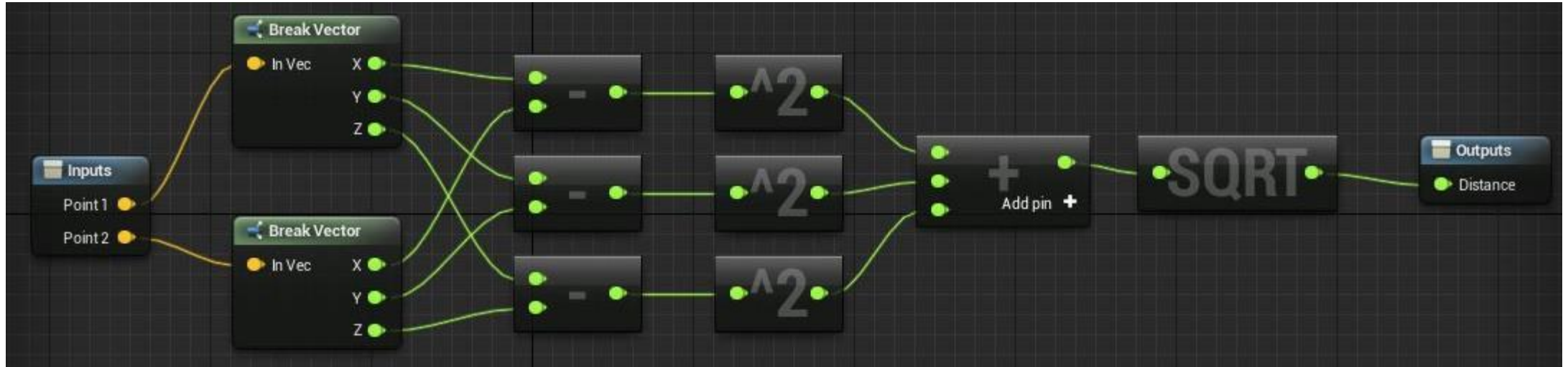


Алгоритмизация и программирование

Лекция 7 (Go)

Язык математиков и инженеров

Процедурное программирование – последовательность вызовов процедур/функций:



Функциональное программирование – набор утверждений:

```
factorial :: Integer -> Integer
factorial 0 = 1
factorial n = n * factorial (n - 1)
```

Язык бизнеса

Клиент открывает депозитный счёт с начальной суммой 1000 рублей:

```
initial_amount = 1000  
open_deposit(client_id, initial_amount) // процедурный стиль  
client.open_deposit(initial_amount)    // ООП
```

Клиент (alex) переводит клиенту (anna) 500 рублей:

```
transfer_money(alex_id, anna_id, 500) // процедурный стиль  
alex.transfer_to(anna, 500)           // ООП
```

Если клиенту разрешён перевод в размере 1000 рублей, то ...:

```
if get_transaction_status(transaction(client_id, 10000, "RUB")) == ALLOWED {...} // процедурный  
if client.transaction(10000, "RUB").is_allowed() {...} // ООП
```

ООП

Как видно, теперь вместо данных простых типов `int`, `float64` и т.д и функций оперирующих этими данными мы хотим использовать программные сущности `client`, `bank` и т.д. которые обладают некоторым собственным набором операций (поведение) и обладают всей необходимой информацией, для осуществления этих операций (состояние).

Такую программную сущность будем называть **объектом**, а тип данных, который описывает объект будем называть **классом**.

Стиль программирования, при котором код состоит из взаимодействующих друг с другом объектов будем называть объектно-ориентированным стилем программирования.

Класс



Объекта класса Кот



Объекта класса Кот



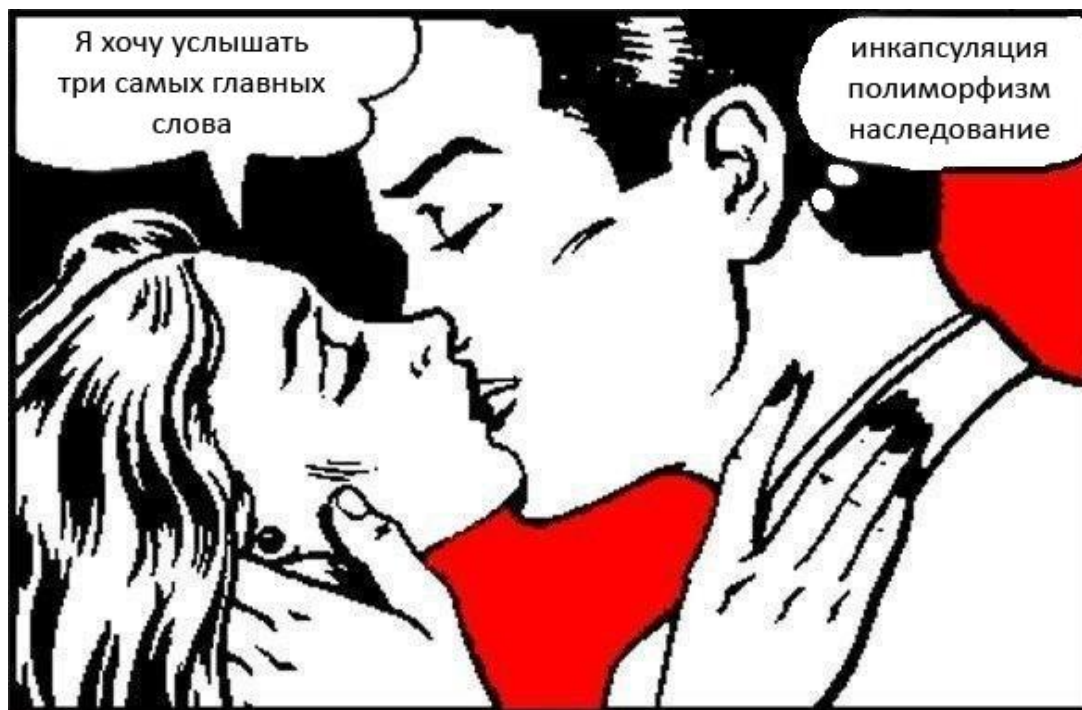
Объекта класса Кот



Основные принципы ООП

ООП – это набор идей и реализация этих идей в разных языках программирования отличается в соответствии с идеологией и возможностями самого языка. Несмотря на это, существует набор принципов которые должен поддерживать язык, чтобы называться ООП языком.

В зависимости от источников, основные принципы ООП отличаются как по количеству, так и по составу, но обычно выделяют 3 основных принципа (часто сюда добавляют ещё абстракцию):



Поддержка ООП в Go

По формальным признакам Go не является объектно-ориентированным языком (как Java или C++), т.к. не поддерживает некоторые принципы ООП в их классическом понимании. Несмотря на это, в Go присутствует всё необходимое, чтобы писать код в ООП стиле.

В Go структуры используются для определения типов данных, которые могут содержать поля и к структурам можно привязывать методы:

```
type Cat struct {  
}  
  
// Метод для структуры Cat  
func (c Cat) SayHello() {  
    fmt.Println("Мур")  
}
```

Состояние объекта при этом описывается при помощи полей структуры, а поведение реализуется при помощи методов.

Какие бывают поля – поля объекта

```
type Point struct{  
    x1 int    // Поле-значение  
    x2 *int   // Поле-указатель  
    Address  // Анонимное поле  
    OnTemperatureChange func(float64) // Поле-функция  
}
```

Тип данных может быть любым, не только `int`. Константных полей не бывает

Какие бывают поля – поля объекта

```
package main

import (
    "fmt"
)

type Point struct {
    x, y int
}

func main() {
    var p1, p2 = Point{1, 1}, Point{2, 2}

    fmt.Printf("p1: [%d, %d]\n", p1.x, p1.y)
    fmt.Printf("p2: [%d, %d]\n", p2.x, p2.y)
}
```

Каждый **объект** имеет собственный, не зависящий от других, набор полей объекта;

Доступ к полям объекта происходит через точечную нотацию с указанием нужного **объекта**.

Какие бывают поля – поля "класса"

Аналогов полей класса (static полей как в C++) в Go нет, поэтому далее будем говорить просто "поле" подразумевая обычное поле структуры.

Если очень хочется, то можно вынести класс в отдельный пакет и воспользоваться переменными/константами уровня пакета:

```
package company
```

```
// Общее для всех экземпляров
```

```
const Name string = "TechCorp"
```

```
var EmployeeCount int = 0
```

```
type Employee struct {
```

```
    name string // поле экземпляра
```

```
    id    int    // поле экземпляра
```

```
}
```


Какие бывают поля – "поля класса"

"./company/company.go"

```
package company

// Общее для всех экземпляров
const Name string = "TechCorp"
var EmployeeCount int = 0

type Employee struct {
    name string // поле экземпляра
    id    int    // поле экземпляра
}

func NewEmployee(name string) *Employee {
    EmployeeCount++ // модифицируем "статическое поле"
    return &Employee{name: name, id: EmployeeCount}
}
```

Доступ к переменным/константам уровня пакета происходит при помощи точечной нотации с указанием названия пакета. Доступ возможен, если идентификатор экспортируем, т.е. начинается с заглавной буквы.

"main.go"

```
package main

import (
    "fmt"
    "project/company" // импортируем наша пакет
)

func main() {
    company.NewEmployee("John")
    company.NewEmployee("Mary")
    fmt.Printf("Название компании: %s; Количество сотрудников: %d", company.Name, company.EmployeeCount)
}
```

Какие бывают методы – методы объекта

В Go есть только обычные методы (не константные и не виртуальные), которые могут отличаться по типу Ресивера (получателя - аналог `this` из C++):

```
type Point struct {  
}  
  
// Метод для Point. На то, что это метод для Point указывает ресивер перед именем метода  
func (p Point) String() string {  
    return "Point"  
}
```

Можно добавлять методы для любых типов, не только для структур:

```
type MyInt int // Свой тип на основе int  
  
// Метод для MyInt  
func (p MyInt) String() string {  
    return "MyInt"  
}
```

При таком объявлении методы не наследуются:

```
type MyPoint Point // Нет методов
```

Тип ресивера при объявлении метода

```
type Point struct {  
    x, y int  
}
```

// Ресивер по значению

```
func (p Point) GetX() int {  
    return p.x  
}
```

// Ресивер по указателю

```
func (p *Point) SetX(x int) {  
    p.x = x  
}
```

В Go нет перегрузки, в том числе и по типу ресивера;
Для единообразия все методы типа часто объявляют с одним типом ресивера (обычно по указателю);
Имя ресивера обычно соответствует первой букве типа.

Работает с копией объекта;
Изменения в методе не влияют на оригинал.

Работает непосредственно с объектом;
Может модифицировать поля;
Эффективнее для больших (> 64 байт) структур (нет копирования);

Метод объекта – вызов метода

```
package main

import "fmt"

type Point struct {
}

func (p *Point) simpleMethodPtr() {
    fmt.Println("simpleMethodPtr")
}

func (p Point) simpleMethod() {
    fmt.Println("simpleMethod")
}

func main() {
    var p Point
    p.simpleMethod()
    p.simpleMethodPtr() // Go автоматически берет адрес значения при вызове метода с ресивером по указателю
    Point{}.simpleMethod()
    Point{}.simpleMethodPtr() // Ошибка

    var pPtr *Point = &Point{}
    pPtr.simpleMethod() // Go автоматически разыменовывает указатель при вызове метода с ресивером по значению
    pPtr.simpleMethodPtr()
    (&Point{}).simpleMethod()
    (&Point{}).simpleMethodPtr()
}
```

Доступ к методам происходит через точечную нотацию, независимо от ресивера, с указанием **объекта (указателя на объект)** у которого вызывается метод.

Метод объекта – вызов метода через интерфейс

```
package main

import "fmt"

type Printer interface {
    ToString() string
}

type Point struct {
}

// Ресивер по указателю
func (p *Point) ToString() string {
    return "Point"
}

func main() {
    var p Point
    var pPrt *Point = &Point{}

    var priner Printer = p // Ошибка
    fmt.Println(priner.ToString())

    var priner2 Printer = pPrt
    fmt.Println(priner2.ToString())
}
```

Метод привязывается к ресиверу.
В данном случае у Point нет методов, а у *Point, есть метод ToString() string поэтому p НЕ реализует интерфейс Printer.

Метод объекта – аналог виртуальных методов

В C++ виртуальные методы обеспечивают возможность динамического полиморфизма. В Go такой функционал обеспечивается при помощи интерфейсов, поэтому в Go нет необходимости в наличии виртуальных методов.

```
type Writer interface {  
    Write([]byte) (int, error)  
}
```

```
func foo(w Writer){ /* ... */ }
```

```
type File struct { /* ... */ }
```

```
func (f *File) Write(data []byte) (int, error) { /* ... */ }
```

```
type Buffer struct { /* ... */ }
```

```
func (b *Buffer) Write(data []byte) (int, error) { /* ... */ }
```

Если передать в функцию foo, как аргумент, экземпляр *File, то будет вызван его метод Write, если *Buffer, то его метод. Т.е. реализация метода Write не зафиксирована статически, а будет определяться динамически в зависимости от фактического типа переданного объекта.

Методы объекта – доступ к другим членам класса из тела метода

```
type Point struct {  
    x, y int  
}  
  
func (p *Point) GetY() int {  
    return p.x  
}  
  
func (p *Point) ToString() string {  
    return fmt.Sprintf("X: %d; Y: %d", p.x, p.GetY())  
}
```

В теле метода можно получить доступ к любым полям и методам структуры.

Какие бывают методы – "методы класса"

Аналогов методов класса (static методов как в C++) в Go нет, поэтому далее будем говорить просто "метод" подразумевая обычный метод структуры или другого пользовательского типа.

Если очень хочется, то можно вынести класс в отдельный пакет и воспользоваться простыми функциями уровня пакета:

```
package company

// Общее для всех экземпляров
const Name string = "TechCorp"

func GetName() string{
    return Name
}
```

Какие бывают методы – "методы класса"

```
package company                                     "./company/company.go"

// Общее для всех экземпляров
const Name string = "TechCorp"
var EmployeeCount int = 0

func GetName() string{
    return Name
}

type Employee struct {
    name string // поле экземпляра
    id   int    // поле экземпляра
}

func NewEmployee(name string) *Employee {
    EmployeeCount++ // модифицируем "статическое поле"
    return &Employee{name: name, id: EmployeeCount}
}
```

Доступ к функциям уровня пакета происходит при помощи точечной нотации с указанием названия пакета. Доступ возможен, если идентификатор экспортируем, т.е. начинается с заглавной буквы.

```
package main                                           "main.go"

import (
    "fmt"
    "project/company" // импортируем наша пакет
)

func main() {
    fmt.Printf("Название компании: %s", company.GetName())
}
```

Специальные методы

Конструктор

Конструктор

```
type User struct {  
    name string  
    email string  
}  
  
// Аналог конструктора  
func NewUser(name, email string) *User {  
    return &User{  
        name: name,  
        email: email,  
    }  
}  
  
// Аналог второго конструктора  
func NewAdminUser(email string) *User {  
    return &User{  
        name: "admin",  
        email: email,  
    }  
}  
  
func main() {  
    user := NewUser("Alice", "alice@example.com")  
    admin := NewAdminUser("admin@example.com")  
}
```

В Go нет понятия – конструктор, вместо этого, за инициализацию полей структуры отвечают специальные фабричные функции, у которых в качестве возвращаемого значения указывается либо экземпляр, либо указатель на экземпляр структуры;

Эти функции, обычно, начинаются с New (с заглавной буквы) и далее идёт имя структуры.

Т.к. в Go отсутствует перегрузка, то при необходимости иметь несколько "конструкторов" используются разные имена.

Деструктор

Деструктор

```
type DatabaseConnection struct {  
    conn    *sql.DB  
    closed bool  
}  
  
func (db *DatabaseConnection) Close() error {  
    if db.closed {  
        return nil  
    }  
    db.closed = true  
    return db.conn.Close()  
}  
  
// Использование с defer  
func processData() error {  
    db := &DatabaseConnection{}  
    defer db.Close() // Гарантированное закрытие  
  
    // Работа с базой данных  
    return nil  
}
```

В Go нет понятия деструктор – вместо этого, за корректное освобождение ресурсов отвечает сборщик мусора, и при необходимости, специальный метод (обычно `Close`, но может быть любым) вызванный при помощи `defer`;

Оператор **defer** обеспечивает выполнение отложенных функций даже в случае ошибки или вызова функции **panic**, гарантируя освобождение ресурсов.

Деструктор - паттерн "WithCleanup"

```
type FileHandler struct {
    file *os.File
}

func (fh *FileHandler) Close() error {
    if fh.file != nil {
        return fh.file.Close()
    }
    return nil
}

func WithFile(filename string, handler func(*FileHandler) error) error {
    fh := &FileHandler{}
    var err error
    fh.file, err = os.Open(filename)
    if err != nil {
        return err
    }
    defer fh.Close() // Автоматическая очистка

    return handler(fh)
}

// Использование
err := WithFile("data.txt", func(fh *FileHandler) error {
    // Работа с файлом
    return nil
})
```

Внимание! Оператор **defer** отработает только при завершении функции, но не при выходе из текущей области видимости.

Чтобы не откладывать освобождение на потом можно воспользоваться лямбда функцией или создать аналог Python конструкции **with**:

```
with open('file.txt', 'r') as file:
    content = file.read()
```

Деструктор - финализатор

```
import "runtime"

type Resource struct {
    data []byte
}

func NewResource() *Resource {
    r := &Resource{data: make([]byte, 1024)}

    // Установка финализатора
    runtime.SetFinalizer(r, func(r *Resource) {
        fmt.Println("Resource finalized")
        // Очистка ресурсов
    })

    return r
}
```

Когда сборщик мусора Go готов собрать объект, оставшийся без ссылок, предварительно вызывается финализатор.

Обычно не рекомендуется использовать финализаторы, а в случае использования обязательно документировать этот факт.

Финализаторы могут быть полезны при работе с кодом выделяющим память вручную, например с библиотеками языка C.

Конструктор копирования

Конструктор копирования

```
type ComplexStruct struct {
    Name      string
    Scores    []int
    Metadata  map[string]string
}

// Метод для глубокого копирования
func (cs *ComplexStruct) DeepCopy() *ComplexStruct {
    copy := &ComplexStruct{
        Name: cs.Name,

        // Копирование слайса
        if cs.Scores != nil {
            copy.Scores = make([]int, len(cs.Scores))
            copy(copy.Scores, cs.Scores)
        }

        // Копирование карты
        if cs.Metadata != nil {
            copy.Metadata = make(map[string]string)
            for k, v := range cs.Metadata {
                copy.Metadata[k] = v
            }
        }

        return copy
    }

    // Фабричный метод для создания копий
    func CopyComplexStruct(original *ComplexStruct) *ComplexStruct {
        return original.DeepCopy()
    }
}
```

Понятия конструктор копирования в Go не существует. Для простого (побайтового) копирования используется обычное присваивание. Для глубокого копирования стандартный способ отсутствует.

Часто "создадут" собственные методы для правильного копирования, которые называют DeepCopy или Clone.

Существует ряд пакетов помогающий в этом вопросе:
<https://habr.com/ru/companies/avito/articles/743332/>

Конструктор перемещения

Конструктор перемещения

Понятия конструктор перемещения в Go не существует. В Go отсутствуют механизмы при помощи которых можно было бы реализовать функционал подобный конструктору перемещения.

Спецификаторы доступа

Спецификаторы доступа

В Go отсутствуют спецификаторы областей видимости аналогичные спецификаторам C++ (`private`, `protected`, `public`). Вместо этого используется система контроля доступа, основанная на **регистре первой буквы** идентификатора.

Второй механизм – интерфейсы, которые предоставляют доступ только к методам и только к тем, которые описаны в интерфейсе.

В третьих можно использовать контроль доступа через пакетную организацию. Т.е. поместив пакеты в каталог "internal" можно сделать их доступными только внутри текущего пакета и подпакетов.

Управление доступом – регистр первой буквы

```
package mypkg
```

```
// Публичная структура
```

```
type User struct {  
    Name    string // Публичное поле  
    Email   string // Публичное поле  
    secret  string // Приватное поле  
}
```

```
// Публичный метод
```

```
func (u *User) GetName() string {  
    return u.Name  
}
```

```
// Публичная функция
```

```
func NewUser(name string) *User {  
    return &User{Name: name}  
}
```

```
// Публичная константа
```

```
const MaxUsers = 100
```

```
// Приватный метод
```

```
func (u *User) setName(name string) {  
    u.Name = name  
}
```

```
// Приватная функция
```

```
func validateEmail(email string) bool {  
    return strings.Contains(email, "@")  
}
```

```
// Приватная переменная
```

```
var userCount int = 0
```

Управление доступом – интерфейсы

```
package storage

// Публичный интерфейс
type Storage interface {
    Save(key string, data []byte) error
    Load(key string) ([]byte, error)
}

/*****/

// Приватная реализация
type fileStorage struct {
    basePath string
}

func (fs *fileStorage) Save(key string, data []byte) error {
    // реализация
    return nil
}

func (fs *fileStorage) Load(key string) ([]byte, error) {
    // реализация
    return nil, nil
}

// Фабричная функция возвращает интерфейс, скрывая реализацию
func NewFileStorage(path string) Storage {
    return &fileStorage{basePath: path}
}
```

Управление доступом – пакетная организация

```
package main                                     "project/main.go"

import (
    "project/auth"
    "project/auth/internal/hasheer" // Ошибка
)

func main() {
    auth.Print()
}
```

```
// Структура:
// project/
//   ├── go.mod
//   ├── main.go
//   └── auth/
//       ├── internal/
//           ├── hasheer/
//               └── hasheer.go
//       └── auth.go
```

```
package auth                                     "project/auth/auth.go"

import (
    "fmt"
    "project/auth/internal/hasheer"
)

func Print() string {
    return fmt.Sprintf(hasheer.NewHasher("123"))
}
```

```
package hasheer                                 "project/auth/internal/hasheer/hasheer.go"

// Доступно только внутри auth и его подпакетов
type Hasher struct {
    secret string
}

func NewHasher(secret string) *Hasher {
    return &Hasher{secret: secret}
}

func (h *Hasher) Hash(password string) string {
    return "hashed:" + password
}
```

Неполный тип

Неполный тип

В Go нет прямого аналога неполных типов (`incomplete types`) из C++, т.к. область видимости идентификаторов объявленных внутри пакета – весь пакет, а идентификаторы импортируемые из других пакетов импортируются в самом верху файла, т.е. нет такой точки выше которой идентификатор не виден, а ниже которой виден.

Использование себя как поля

// Нельзя

```
type Node struct {  
    data int  
    next Node // Следующая нода  
}
```

// Так можно

```
type Node struct {  
    data int  
    next *Node // Следующая нода  
}
```

Структура не может использовать себя в качестве своего же поля, потому что это приведёт к бесконечному рекурсивному выделению памяти под объект такой структуры;

Структура может использовать указатель на себя как поле, как возвращаемый тип или параметр метода.

Решение циклических зависимостей

```
type A struct{  
    b_field B  
}
```

```
type B struct{  
    a_field A  
}
```

```
/****** компилятор разрешает циклические зависимости *****/
```

```
type A struct{  
    b_field *B  
}
```

```
type B struct{  
    a_field *A  
}
```

Использование в методах

```
package main

import "fmt"

type A struct {
}

func (a *A) method(param B) {
    fmt.Println(param.value)
}

type B struct {
    value int
}

func main() {
    var a A
    var b = B{10}

    a.method(b) // 10
}
```


Устройство объекта в памяти

Объект в памяти

```
package main

import (
    "fmt"
    "unsafe"
)

type A struct{}

type B struct {
    value int
}

type C struct {
    value int
}

func (c *C) String() string {
    return fmt.Sprint(c.value)
}
```

```
func main() {
    a := A{}
    b := B{}
    c := C{}

    fmt.Printf("A: %d\n", unsafe.Sizeof(a)) // A: 0
    fmt.Printf("B: %d\n", unsafe.Sizeof(b)) // A: 8
    fmt.Printf("C: %d\n", unsafe.Sizeof(c)) // A: 8
}
```

Место в экземпляре структуры занимают только поля;
Методы не занимают в памяти объекта никакого места.

Разделение кода класса на файлы

Разделение кода на файлы

В Go код можно разделять на файлы. Если код принадлежит одному пакету, то область видимости идентификаторов распространяется на все файлы.

```
package bank

type Account struct {
    balance int
    owner   string
}

// Конструктор
func NewAccount(owner string) *Account {
    return &Account{
        owner:  owner,
        balance: 0,
    }
}
```

```
package bank

func (a *Account) Deposit(amount int) {
    if amount > 0 {
        a.balance += amount
    }
}

func (a *Account) Withdraw(amount int) bool {
    if amount > 0 && a.balance >= amount {
        a.balance -= amount
        return true
    }
    return false
}
```

Перегрузка операторов

Перегрузка операторов

```
package main

import (
    "fmt"
    "sort"
)

type Person struct {
    Name string
    Age  int
}

type ByAge []Person

func (a ByAge) Len() int      { return len(a) }
func (a ByAge) Swap(i, j int) { a[i], a[j] = a[j], a[i] }
func (a ByAge) Less(i, j int) bool { return a[i].Age < a[j].Age }

func main() {
    people := []Person{
        {"Алексей", 30},
        {"Мария", 25},
        {"Иван", 35},
    }

    sort.Sort(ByAge(people))
    fmt.Println(people) // [{Мария 25} {Алексей 30} {Иван 35}]
}
```

В Go напрямую нет перегрузка операторов, вместо этого обычно используются методы с соответствующими именами и/или интерфейсы.

Оператор преобразования типа

В Go нет прямого аналога преобразования типов для этих целей, как правило используются либо стандартные преобразования либо собственные методы.